

CCN-WSN – a lightweight, flexible Content-Centric Networking Protocol for Wireless Sensor Networks

Zhong Ren ^{#1}, Mohamed A. Hail ^{*2}, Horst Hellbrück ^{*3}

[#] *Computer Science, Yale University*

51 Prospect Street, New Haven, CT 06511, United States

¹ zhong.ren@yale.edu

^{*} *Electrical Engineering and Computer Science, Lübeck University of Applied Sciences*

23562 Mönkhofer Weg 239, Lübeck, Germany

²hail,³hellbrueck}@fh-luebeck.de

Abstract—In future Internet research, content centric networking (CCN) is a new promising approach. CCNx has been introduced recently as an open source protocol suite for CCN and implementation base for practical research. In wireless sensor networks (WSNs) research, data or content centric approaches like in-network processing and data aggregation are important. While the principle of CCN is a suitable approach in WSNs, the CCNx protocol suite designed for PCs is not applicable to resource-constrained WSNs. Therefore, we design, implement and evaluate a lightweight variant of a CCN protocol specifically for WSNs called CCN-WSN as a lightweight alternative to implementing IP for sensor networks. Key concepts of CCNx protocol are integrated but a variety of aspects are revised to meet the memory and computational constraints of sensor nodes and communication patterns in WSNs. E.g. the message format is simplified and some fields are omitted completely. Instead, we propose a flexible naming strategy which extends the functionality of content names to add small amount of data in interest messages. For performance evaluation a challenging time-synchronization application was implemented with CCN-WSN to demonstrate the flexibility of the approach and a comparison with a reference protocol for data dissemination called AutoCast is presented.

I. INTRODUCTION

Content-centric networking (CCN), which is also called content-based networking, data-oriented networking or named data networking, is a proposal for a new architecture of computer networks to replace today's node or host centric networking. The Internet protocol (IP) created in the 1970s is still the dominant networking protocol today. IP addresses identify hosts by so called IP addresses in a hierarchical addressing scheme similar to the zip code for mail. Transport protocols like transmission control protocol TCP and user datagram protocol UDP are based on IP service including IP addresses. Users prefer names instead of IP addresses (numbers). Therefore, the domain name system (DNS) as a supporting technology maps between names and IP addresses. If we consider user requirements in more detail, users are interested in content and not the infrastructure. Consequently, today communication network architecture needs a re-design to avoid users and application programmers having to reference a specific location where data is to be retrieved from. Content-centric networking is a promising architectural approach to

support these network usage patterns. The benefits of CCN are content caching, self-adaptation, and built-in security at data level. CCNx [1] is an open source project that enables significant experiments to begin in the research field of CCN. CCNx replaces named hosts with named content in the sense of Palo Alto Research Center's CCN architecture [2] as a major, long-term research and development effort.

In WSNs networks, usage pattern are similar than in today's Internet, where users and application programmers are interested in content mainly instead of sensor nodes. Challenges in WSN also include intelligent caching, self-adaptation as well as security and are far from being solved. Instead of following the traditional approach using 6LoWPAN, RPL and CoAP, we skip this step and propose a lightweight CCN protocol implementing key aspects of CCNx with the lessons learned from WSN research. With such an approach we are able to integrate WSNs into CCNx networks easily in future with the help of straightforward, simple gateways.

The contributions of the paper are as follows:

- We design a lightweight and flexible alternative of the CCNx protocol designed specifically for WSNs called CCN-WSN protocol
- We implement the protocol using the template algorithmic library Wiselib [3] on a sensor network platform.
- We implement a challenging application showing the flexibility of CCN-WSN
- We conduct performance evaluation against a general purpose data dissemination protocol AutoCast [4]

The rest of this paper is organized as follows: Section II discusses related work in the field. We introduce our approach in the next section and implementation in Section IV. An evaluation of CCN-WSN is presented in Section V. Section VI concludes the paper and presents future work.

II. RELATED WORK

Recently, the content centric networking (CCN) paradigm has gained a lot of attention in future network research also in wireless sensor network (WSN) community. In this section we will compare our approach with related work in the field of WSNs and CCN. First, we discuss solutions for locating

and routing of content based on names or semantics. In the second part of this section related work in the field of CCN using naming scheme and energy efficiency of CCN protocols is introduced. Finally, we discuss related implementations of CCN protocols in WSNs.

Efficient location and routing of content based on names and semantics has been addressed by many publications recently for future Internet research. The Data-Oriented Network Architecture (DONA) [5] proposes administrative access policies and uses flat, self-certifying names to allow routing directly on flat names. DONA introduced resolution handlers (RHs) as alternative to existing DNS servers. The name resolution is performed by two basic commands. The command *FIND* locates the object *NAME* whereby the RHs return the location of a nearby copy if available. The command *REGISTER* adds discoverable data to the system. RHs keep a registration table to map a name to a next-hop RH and distance to destination. TRIAD [6] suggests a route-by-name to find the closest content/copy of the request. ROFL [7] proposes to route data directly based on semantics of flat names and creates a circular namespace for correct routing, e.g., Distributed hash table (DHT). All aforementioned approaches do not provide libraries and implementation to build or run applications for experimental and research purposes. We decided to implement CCN protocol on WSN because protocol specifications, reference implementation and documentation of CCNx are released to enable collaboration with researchers.

Content-Centric Networking (CCN), as introduced in Section I, has been proposed and described in [8]. CCNx [1] is a reference implementation of CCN providing location-independent delivery services for named data. Named data packets may be cached at CCN nodes or routers. The similar architecture has emerged in multiple proposals, including the Network of Information (NetInf) [9], the Publish-Subscribe Internet Routing Paradigm (PSIRP) [10]. Although these designs and implementations provide flexibility, security and performance for the future Internet, they are not applicable to the resource-constrained WSNs. As the authors in [11] argue that CCN protocols can be implemented efficiently, energy saving plays an important role in CCN itself. We implement the CCN protocol on our WSN nodes to investigate energy efficiency of CCN.

In [12] the authors suggest an implementation of CCN for emergency wireless Ad-hoc environment (MANET CCN). MANET CCN is based on CCN and supports a naming scheme to distribute the content in mobile Ad-hoc networks (MANET). In contrast to our target platform, the authors implement MANET CCN on laptop class devices running Linux OS and do not focus on WSN. In our approach, we designed, implemented and evaluated a lightweight variant of CCN protocol specifically for wireless sensor nodes which have severe constraints in memory and computational power. To the best of our knowledge, the CCN-WSN protocol is the first protocol that distributes content or requests based on flexible naming scheme in WSNs.

IPv6 over Low power Wireless Personal Area Networks

(6LoWPAN) is the current approach to connect WSNs to the Internet. At the same time, the Constrained Application Protocol (CoAP) provides web service functionality on top of User Datagram Protocol (UDP) of WSNs. We foresee that using CCN straight away on top of IEEE 802.15.4 provides an efficient solution and reduces overhead.

III. PROTOCOL DESIGN

This section presents the design of our CCN-WSN protocol which is based on ideas of the original CCNx protocol [1]. For details of CCNx we refer to [1]. Due to space restrictions we describe only differences between messages of the protocols, the node model and our flexible naming scheme.

A. Message Format

In CCN-WSN as well as in CCNx, communication is based on two message types. When a node requests content, it broadcasts an *Interest* (similar to HTTP GET request) to the network. If a node receives the interest and has the requested content, it sends a *Content Object* (similar to HTTP response), following the “footprint” of the interest back to the requesting node. Both interest and content object contain a *Content Name* that specifies the requested or transmitted content. Unlike some common protocols such as IP, CCNx does not have fixed-length fields in messages. The message format is defined by XML schema with each field of arbitrary length. Due to space restrictions we refer to [13] for a full explanation of the message format of CCNx. Herein we explain the necessary modifications made for CCN-WSN.

The flexible design and usage of XML in CCNx are not applicable to WSNs with limited bandwidth and computational power. In particular, the maximum length in IEEE 802.15.4 is 127 octets. Consequently, we redesign the message format in CCN-WSN for IEEE 802.15.4 in WSNs. Most of the fields in CCN-WSN are fixed-length while some fields are omitted completely. In addition, encoding is simplified using straightforward serialization.

Message Type, shortened to 1 byte in CCN-WSN, identifies whether the received message is an interest or a content object as shown in Figure 1 and Figure 2. A **Content Name** is hierarchically structured, composed of a sequence of *Components* to identify content. To fit into an IEEE 802.15.4 frame, the maximum length of a content name is 50 octets and limited to five components with maximum 15 octets each (see Table I for details). The length of the variable-length name is specified in **Name Length** (1 byte) of the message. We will introduce the naming strategy in more detail at this section’s end.

For content objects, each content name is bound to a single piece of data and the binding is immutable. For interest messages, however, a content name may specify either a chunk or a collection of data. E.g. an interest with the name */domain.com/temperature* may request any content object whose name starts with this prefix. For this reason, the content name in an interest is also referred to as a *prefix* in the sense of longest prefix match used in routing protocols.

Item	Name Length	Component Length	Number of Components
Maximum Value	50	15	5

TABLE I: Constraints on the Content Name in CCN-WSN

In CCNx, an *Interest Message* comprises of content name, selector and nonce. Figure 1 illustrates the differences between interest messages in CCNx and CCN-WSN.

	CCNx Protocol	CCN-WSN Protocol
	Message Type	Message Type
	-	Name Length
	Content Name	Content Name
Selector	Min Suffix	-
	Max Suffix	-
	Publisher Public Key	-
	Exclude	-
	Child Selector	Child Selector
	Answer Origin Kind	-
	Scope	-
	Interest Life Time	Interest Life Time
	Nonce	-

Fig. 1: Interest Message Format for CCNx and CCN-WSN

The field **Selector** in the CCNx protocol contains a number of nested fields, which qualify the content object that matches the interest. Considering simpler communication pattern and limited frame size in WSNs, the CCN-WSN protocol includes two essential parts: **Child Selector** and **Interest Life Time**. Missing fields are compensated to some extent by our flexible naming strategy based on hierarchical content names. The **Child Selector** (1 byte) provides additional selection criteria for content in the case of multiple matching for a given interest. For a value of 0, the leftmost child of the prefix in the name tree will be preferred and rightmost for a value of 1. Leftmost and rightmost herein refer to the least and greatest components according to a lexicographical order defined in the protocol. Default value is 0 (leftmost). The field **Interest Life Time** (2 bytes) specifies the time interval before interest times out. Applications on resource-constrained devices in WSNs require this field to delete outdated content and interests. Default value is 4 seconds.

To distinguish same interests from different nodes in the network and to prevent looping of interest, the CCNx protocol defines a nonce to carry a randomly-generated byte string for detecting duplicates. In WSNs, however, each sensor node typically has a single radio interface and one or several applications as available communication paths (see next section for details). Hence, we assume that interest messages cannot loop in CCN-WSN and **Nonce** is omitted in CCN-WSN.

A *Content Object* consists of four major parts: name,

signature, signed info and the content itself in CCNx. CCN-WSN simplifies the format by omitting signature and some sub-fields in signed info as shown in Figure 2. Message type, name length and name have been discussed above.

	CCNx Protocol	CCN-WSN Protocol
	Message Type	Message Type
	Signature	-
	-	Name Length
	Content Name	Content Name
Signed Info	Publisher Public Key	-
	Timestamp	-
	Type	Content Type
	Freshness Seconds	Freshness Seconds
	Final Block ID	Final Block ID
	Key Locator	-
	-	Content Length
	Content	Content

Fig. 2: Content Objects Format for CCNx and CCN-WSN

In CCN-WSN **Signature** and all security-related fields are omitted, because security and privacy are not in the scope of our current design.

Timestamp specifying the creation time of content is omitted too. We prefer timestamps in the content name as a versioning component and be processed more flexibly in CCN-WSN. The time synchronization application in Section V demonstrates flexible use of timestamp in content names.

Content Type (1 byte) specifies the type of content in this content object. In the original CCNx protocol, types include DATA (normal), ENCR (encrypted), NACK (acknowledgement), etc. Although content types in CCN-WSN are not yet defined, we keep this field for future use.

Freshness Seconds (2 bytes) supports content updates and **Final Block ID** (1 byte) content segmentation as in the original CCNx protocol. **Content Length** (1 byte) provides the length of the payload in the content object.

B. Node Model

Sensor nodes in the CCN-WSN protocol follow the node model defined in the CCNx protocol, as shown in Figure 3 and explained in more detail in [1]. Messages are received and sent through *Faces*. A face is a generalization of the concept interface and is either a network interface or an interface to a local application. Specific connection details are encapsulated in the face implementation itself. For example, a sink node in a WSN is connected to the WSN via IEEE 802.15.4 and a LAN via Ethernet. In addition, an application is also running on the sink node to process the sensed data. Accordingly, this node would have three distinct faces (cf. Figure 3).

The *Content Store* (CS) is the first table in Figure 3 caching received content for later use. Apart from the content itself, arrival time of each content object is stored in the CS for

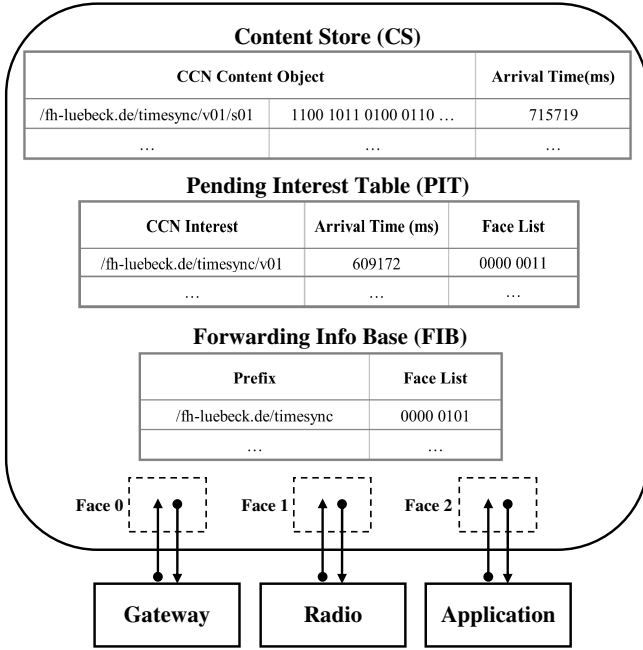


Fig. 3: Node Model in the CCN-WSN Protocol

staleness handling. As resource-constrained sensor nodes can only maintain a very limited number of entries, CCN-WSN adopts a rather “lazy” replacement policy compared to least recently used (LRU) policy of CCNx. The daemon deletes oldest content when number of entries reaches a threshold.

The second table named *Pending Interest Table (PIT)* in Figure 3 stores interests which have been forwarded but have not been satisfied by matching content. For each interest, the PIT stores a list of faces from which this interest has been received, so that a matching content object can be immediately forwarded to these faces. Arrival time is also stored to handle stale interests. In CCN-WSN, the PIT implements a stringent staleness handling policy.

The *Forwarding Information Base (FIB)* in Figure 3 is comparable to a routing table in IP protocol and contains forwarding entries for interests based on content names. In contrast, each prefix in a FIB can correspond to several faces. Interests matching an entry according to the *longest prefix match* algorithm in the FIB, will be forwarded to all corresponding faces. Otherwise interests are forwarded to a broadcast-capable face instead.

C. Naming Strategy

According to the pull-based communication pattern of CCN, all content or data is sent in response to an interest. However, applications on WSNs generally require both pull-based and push-based communication. E.g. a node detecting an abnormal value will report it to the sink. Moreover, the data transmitted in a WSN could be quite small (e.g. a temperature value of 2 bytes). Therefore, we propose a flexible naming strategy for CCN-WSN to integrate arbitrary data in the interest message

as a name component.

For convenient processing, one byte is defined as a marker and separator for attached data. Based on this naming rule, one name component in the content name may serve as a predicate to further specify a matching content. For example, an interest in a temperature monitoring application may have the following name: /domain.com/testbed/temperature/g{0x1F7A}, where the marker ‘g’ is interpreted by the application as “greater than”. Consequently, only nodes with temperature values greater than the value specified in the interest will send content. Since arbitrary data can be encapsulated in a content name, the name component can contain the ASCII code of the delimiter. To resolve this problem, the CCN-WSN protocol adopts a simple encoding strategy adapted from the percent-encoding in URI syntax [14]. There are two reserved characters in the content name as delimiters ‘/’ and the escaper ‘%’ itself. The forward slash ‘/’ is encoded as ‘%&’, and the percent sign ‘%’ as ‘%0’. This approach minimizes encoding overhead.

IV. IMPLEMENTATION

This section introduces the Wiselib library [3] in which CCN-WSN protocol is implemented. It also provides an overview of our prototype implementation of CCN-WSN.

Wiselib is a generic algorithm library specially designed for wireless sensor networks. It provides a collection of algorithms for WSNs that run on heterogeneous hardware and software platforms. Wiselib is implemented using C++ templates, which is the same technique employed by Standard Template Library (STL) [15] and Boost [16]. The fundamental design of Wiselib is composed of *concepts* and *models* [17]. A concept is a description of a certain entity used in WSNs, which defines abstract data types and signatures of functions. A model is a particular implementation of a concept, which specifies the data types and provides body of the functions. Each hardware or software platform needs to define their own model of the concepts they support. Thereby, the use of Wiselib ensures heterogeneity and generality of our CCN-WSN implementation.

A simplified class diagram in Figure 4 shows the architecture of CCN-WSN. The class *Daemon* is the core class that manages CS, PIT and FIB tables, faces, and processes messages. Each table is implemented as a class, which contains a list of the corresponding data structures and implements the necessary functions to be called by the *Daemon*. For example, the *PIT* holds a list of interest entries, and supports the operations to search for interests, add a new interest, and return the face(s) where a matching content object should be sent. Besides the three tables, the *Daemon* manages a face table containing references to faces. Each face registers a callback function in the face table and declares whether it is capable of handling broadcasts. For convenience, the *Daemon* allocates a unique face ID to each face. Interests and content objects are processed in the *receive(data)* function of the *Daemon*.

The daemon matches incoming messages with the corresponding tables, and calls `send(data)` or `broadcast(data)` based on the processing result, which in turn calls the callback function of the appropriate faces.

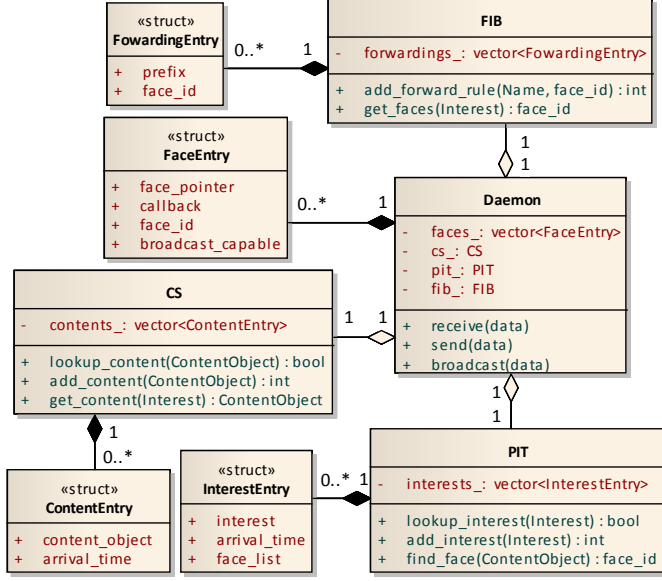


Fig. 4: Class Diagram of a Node in CCN-WSN

For implementation of faces we define a `BaseFace` class comprising a reference to the `Daemon` in order to send a message to it. This class also includes an abstract function `bind_ccn(Daemon)` to register a callback function to the `Daemon` so that it can receive messages from the `Daemon`. Each face implementation extends the `BaseFace` class and overrides the `bind_ccn(Daemon)` function. This design ensures flexibility and extensibility of faces in CCN-WSN.

V. EVALUATION

In this section we evaluate our CCN-WSN implementation. We describe the implementation of a time synchronization application and compare CCN-WSN's performance with AutoCast [4] data dissemination protocol.

Time synchronization in wireless sensor networks is important for collaboration and coordination of sensor nodes. In this work, we implemented Lamport timestamps [18] to evaluate the CCN-WSN protocol. The algorithm for our implementation is straightforward with each sensor node in the WSN running a physical clock which is not very precise. Every five seconds, nodes broadcast their logical time which is derived from the physical clock and an offset. A node receiving a timestamp that is larger than its local logical time will adjust its local time to the newly arrived timestamp.

For reference, we choose an existing implementation based on AutoCast [4], a data dissemination protocol. In AutoCast, each node broadcasts data units containing timestamps across the network. The handling of received timestamp is based on the state of the data unit. If the timestamp is larger than the logical time, the data unit is `ACTIVE` and will be further

disseminated. Otherwise, the data unit is `STALE` and will not be propagated. More details can be found in [4].

We have chosen time synchronization as application as it is challenging for the CCN paradigm. The original ideas of CCN do not foresee time synchronization application. In CCN pull mechanism, timestamps will be sent as content as a response to an interest. This approach is inefficient since requesting timestamps via interest increases the number of messages and causes unnecessary bandwidth usage.

Therefore, we propose a more effective approach to implement a *bi-directional* time synchronization using the flexibility of the naming scheme. As presented in Section III, arbitrary data can be encapsulated in a name component. An example of a content name for time synchronization in an interest is: `/domain.com/testbed/timesync/t{0x23e4a56b}`. The marker 't' specifies that this name component is a timestamp in the interest followed by the value in milliseconds in hex. When receiving such an interest, nodes will extract the timestamp and adjust their logical time if needed as mentioned above. If the received timestamp is smaller, however, a content object with the local timestamp is sent back. Since the interest is still in the PIT of the original sender, CCN daemon will forward the content to the application and the larger timestamp will be updated. Accordingly, in this application each interest serves two purposes simultaneously, i.e., advertising its own timestamp and requesting a larger one if available.

We implemented the time synchronization application for both protocols on our WSN testbed, which comprises six sensor nodes (SN) deployed in separate rooms communicating via IEEE 802.15.4, as shown in Figure 5. Each sensor node is based on an Atmel ATxmega128A1 microcontroller with a clock frequency of 32 MHz, 128 KB flash memory, and additional 64 KB RAM. The RF chip is AT86RF231. Although this network is relatively small, there are up to four hops between nodes (i.e. SN1 to SN6).

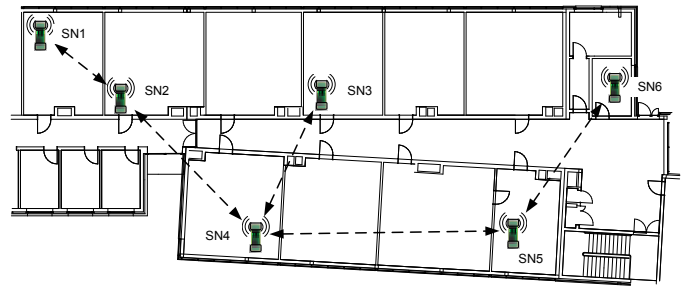
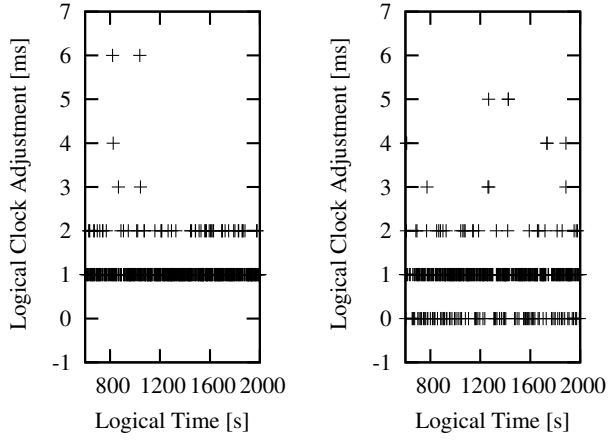


Fig. 5: WSN Testbed for Evaluation

For evaluation, we allowed the system to start up in the first 600 seconds and collected data from 600 second to 2000 second, as shown in Figure 6a and Figure 6b. The logical time adjustment refers to the difference between the updated logical time and the local logical time for each node during synchronization. Consequently, the lesser the adjustment is,



(a) Accuracy with AutoCast (b) Accuracy with CCN-WSN

Fig. 6: Time Synchronization Accuracy

the higher the synchronization accuracy. The results revealed that accuracy in both protocols were within two milliseconds with CCN-WSN being slightly better.

Given the comparable accuracy, we evaluated the bandwidth usage of both protocols, i.e. number of messages and number of bytes. Table II presents the summarizing results for all six sensor nodes. In CCN-WSN, 1576 messages (1057 interest messages and 519 content objects) were sent in contrast with 4041 messages in AutoCast. Regarding the number of bytes, despite having more meta-data overhead, CCN-WSN only transmitted 18.9 KB whereas AutoCast sent 28.3 KB. The results demonstrate a higher bandwidth efficiency of CCN-WSN implementation compared with AutoCast.

The main reason for the better performance of CCN-WSN is that applications control routing of the daemon based on the content of single messages more efficiently. The application layer and routing layer functionality of CCN-WSN are not separated anymore, which adds flexibility as well as room for optimization to the system.

SN1 – SN6	Interest Messages	Content Objects	CCN Messages	AutoCast Messages
Total	1057	519	1576	4041
Total (Kbytes)	10.6	8.3	18.9	28.3

TABLE II: Bandwidth Usage for AutoCast and CCN-WSN

VI. CONCLUSIONS AND FUTURE WORK

In this paper we motivated the need for a lightweight implementation of the CCNx-Protocol targeted for wireless sensor networks which we named CCN-WSN. We followed the content centric approach and the key implementation aspects of CCNx and designed a flexible and extendable protocol. The evaluation demonstrates that content centric networking designed for the Internet is suited for usage in wireless sensor

networks in a light-weight variant. We predict that writing wireless sensor applications with CCN-WSN is much more intuitive than for other related approaches due to the stringent usage of interest, content and implicit routing within the CCN daemon as well as the face concept. In the future, we will evaluation against 6LoWPAN and CoAP. We foresee that using CCN straight away on top of IEEE 802.15.4 will provide an efficient solution with less overhead. Furthermore, we will add applications and semantic functionality to query wireless sensor networks. To improve the performance we will provide a compression scheme for the content names and implement persistent memory for content. Furthermore, we will add lightweight security measures in the implementation to prevent passive and active attacks against CCN and design and implement a gateway for seamless integration of CCN-WSN into CCNx implementation.

VII. ACKNOWLEDGMENTS

This work was funded by the Federal Ministry of Education & Research of the Federal Republic of Germany (Förderkennzeichen 01BK0906, GLab) and (Förderkennzeichen 17PNT017, DataCast).

REFERENCES

- [1] Project CCNx. [Online] <http://www.ccnx.org/>.
- [2] Palo alto research center incorporated (PARC). [online] <http://www.parc.com/services/focus-area/content-centric-networking/>.
- [3] Tobias Baumgartner, Ioannis Chatzigiannakis, Sndor P. Fekete, Christos Koninis, Alexander Krller, and Apostolos Pyrgelis. Wiselib: A generic algorithm library for heterogeneous sensor networks. 2010.
- [4] S. Ebers, M. A. Hail, Stefan Fischer, and Horst Hellbrück. API for data dissemination protocols - evaluation with autocast. In *The Third World Congress on Nature and Biologically Inspired Computing*. IEEE, 2011.
- [5] Teemu Koponen, Mohit Chawla, Byung-Gon Chun, Andrey Ermolinskiy, Kye Hyun Kim, Scott Shenker, and Ion Stoica. A data-oriented (and beyond) network architecture. *SIGCOMM Comput. Commun. Rev.*
- [6] D.R. Cheriton and M. Gritter. Triad: A new next-generation internet architecture, 2000.
- [7] Matthew Caesar, Tyson Condie, Jayanthkumar Kannan, Karthik Lakshminarayanan, and Ion Stoica. Rofi: routing on flat labels. *SIGCOMM Comput. Commun. Rev.*
- [8] Van Jacobson, Diana K. Smetters, James D. Thornton, Michael F. Plass, Nicholas H. Briggs, and Rebecca L. Braynard. Networking named content. In *Proceedings of the 5th international conference on Emerging networking experiments and technologies*, CoNEXT '09, 2009.
- [9] Bengt Ahlgren, Matteo D'Ambrosio, Marco Marchisio, Ian Marsh, Christian Dannewitz, Börje Ohlman, Kostas Pentikousis, Ove Strandberg, René Rembarz, and Vinicio Vercellone. Design considerations for a network of information. In *Proceedings of the 2008 ACM CoNEXT Conference*.
- [10] Nikolaos Fotiou, George C. Polyzos, and Dirk Trossen. Illustrating a publish-subscribe internet architecture, 2009.
- [11] Uichin Lee, Ivica Rimac, and Volker Hilt. Greening the internet with content-centric networking. In *Proceedings of the 1st International Conference on Energy-Efficient Computing and Networking*.
- [12] Soon-Young Oh, Davide Lau, and Mario Gerla. Content centric networking in tactical and emergency manets. In *Wireless Days'10*.
- [13] CCNx technical documentation. [online] <http://www.ccnx.org/releases/latest/doc/technical/>.
- [14] T. Berners-Lee, R. Fielding, and L. Masinter. Uniform resource identifiers (uri): Generic syntax, 1998.
- [15] A. Stepanov and Meng Lee. The standard template library. Technical report, WG21/N0482, ISO Programming Language C++ Project, 1994.
- [16] Boost c++ libraries. [online] <http://www.boost.org/>.
- [17] Tobias Baumgartner and Alexander Kröller. Generic implementation for heterogeneous tiny artifacts. In *Distributed Self-organized Societies of Tiny Artefacts: Design & Implementation*. 2011.

- [18] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Commun. ACM*, 21(7):558–565, July 1978.